

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Zyvor Argus

Page-by-Page Product Manual

2026-09-02

Page-by-page guides

Each guide follows: Purpose → When to use it → How to get there → What you can do → Related pages.
Every route is also listed in the [complete page index](#).

Api

Page	What it covers
API contract	Validate REST endpoints against an OpenAPI schema (Forge preset available).
Auth & session	Login → reusable session file → logout / expiry / negative auth checks.
Live data	Assert WebSocket / SSE streams and optional live-region updates.

Console

Page	What it covers
Ask Zyra	Citation-first product knowledge Q&A — optional Qdrant hybrid RAG. Read-only; does not mutate cluster state. Aligned with Zyra copilot branding across the Zylvor platform.

Journeys

Page	What it covers
AI test	Describe a goal; the autonomous browser agent drives the app toward it.
Flow test	Multi-step user journey (English or step DSL) with optional login/session, video, and Playwright <code>trace.zip</code> .
HAR record / replay	Record a HAR while browsing routes, or replay the UI against a captured HAR (offline / deterministic network).
Import codegen	Paste Playwright codegen JS/TS and convert it into Zylvor Argus flow steps (optionally run immediately).

Operations

Page	What it covers
Findings	Collected issues from API, auth, live-data, vitals, and audit jobs with export/clear.
QA Runs	History table of QA runs with pass/fail chips and sparkline trends.
Schedules	Turn smoke, audit, ping, TLS, flow, route sweep, API, realtime, or vitals into a 5 min–6 h loop.
Videos	Browse and download recorded journey videos / traces from recent jobs.

Overview

Page	What it covers
Command palette	⌘K / Ctrl-K or header Search — spotlight to jump to any action, panel, or Ask Zyra.

Page	What it covers
Hero status	Cluster/app health banner with pods, replicas, last QA run, pass rate, next schedule, and knowledge (Ask Zyra) stat tile.
Live job panel	Live panel for the running job — macOS Terminal chrome, syntax-colored streaming log, per-test chips, Stop / download.
Pods	Pod cards with phase, restarts, and click-through logs; optional cluster events toggle.
Workloads	Deployment and CronJob strip for the argus namespace (when kube access is available).
Mission Control	Live Mission Control console — Apple-style side rail, dark theme, status hero, workloads, pods, category action panels, schedules, findings, and QA run history.
Login	Mission Control sign-in when <code>DASHBOARD_PASSWORD</code> is set. Uses the same dark/light design system as the dashboard (charcoal default, blue accent buttons). Argus Enterprise uses Keycloak/OIDC with demo accounts <code>demo / demo</code> and <code>ssouser / Sso@321</code> (see the Enterprise SSO chapter in this manual).

Pipeline

Page	What it covers
Create from English	Turn an English description into a one-off test (LLM when keyed, heuristic otherwise).
Discover coverage	Scan repo code/docs for untested routes and pages, then propose coverage.
Generate tests	Generate Playwright tests from a product spec (local path or GitHub).
Run tests	Run ► Smoke (optional grep/shard) or ► Full LangGraph pipeline from local or GitHub specs.

Probes

Page	What it covers
Load test	Simple concurrency load test for latency under pressure.
Uptime ping	HTTP uptime / latency checks across a list of URLs.
Probes	One-shot network & security probes: redirects, headers, cookies, robots, exposed paths, API, sitemap, DNS, CORS, compression.
TLS check	Certificate and DNS health for a hostname.

Quality

Page	What it covers
Site audit	Crawl pages for a11y (axe), broken links, SEO, console, perf, and security headers.
Crawl & test	Crawl an arbitrary site and validate every discovered page.
Flaky check	Re-run the suite N times and surface unstable tests.
Web Vitals	Measure Core Web Vitals (LCP / CLS / INP) with optional device and network throttle.

Security

Page	What it covers
Attack chain	Chains exploit-PoC steps via an LLM planner to confirm a multi-step escalation path — same sandbox and opt-ins as Exploit PoC.
Cloud pentest	Credentialed AWS/GCP/Azure CLI enumeration of a described finding in the sandbox — same opt-ins and credential-reference rules as Host pentest.
CVE lookup	Read-only: fingerprints tech/versions and checks them against OSV.dev. No PoC is generated or run — requires a security engagement.
Exploit PoC	LLM-generated, non-destructive verification of a described finding, executed in a locked-down Kubernetes sandbox — requires an exploit-tier engagement and an execution opt-in.
Host pentest	Credentialed SSH enumeration of a described finding via paramiko in the sandbox — needs a third, independent credentialed-pentest opt-in; credentials are always env-var references, never raw values.
LLM red-team	Attacker/judge adversarial-prompt battery against Ask Zyra — prompt injection, system-prompt leak, excessive agency, jailbreak, PII/secret leak — requires a security engagement.
Misconfig scan	Tech/version fingerprinting, wordlist-driven path discovery, security-header value grading, and DNS hygiene checks — requires a security engagement.
Security engagements	Create/revoke the admin-issued, target-scoped authorization required before running misconfig scan, CVE lookup, or LLM red-team.

Visual

Page	What it covers
Compare URLs	Side-by-side visual diff of two URLs (e.g. staging vs production).
Visual regression	Pixel-compare visual snapshots against baselines; optionally update baselines.
Route sweep	Screenshot many routes (or auto-crawl) and diff against route-sweep baselines.
Screenshot	Capture desktop / tablet / mobile screenshots of any URL.

43 guides. Regenerate: `node scripts/customer-docs/generate-guide-index.mjs` .

API contract

Purpose

Validate REST endpoints against an OpenAPI schema (Forge preset available).

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/api-contract`
- UI: Mission Control → **API** → **API contract** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **API contract**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Auth & session

Purpose

Login → reusable session file → logout / expiry / negative auth checks.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/auth`
- UI: Mission Control → **API** → **Auth & session** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Auth & session**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Live data

Purpose

Assert WebSocket / SSE streams and optional live-region updates.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/realtime`
- UI: Mission Control → **API** → **Live data** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Live data**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Ask Zyra

Purpose

Citation-first product knowledge Q&A — optional Qdrant hybrid RAG. Read-only; does not mutate cluster state. Aligned with **Zyra** copilot branding across the Zyvor platform.

When to use it

- You need grounded answers from ingested manuals, API docs, runbooks, or migration guides
- You want citations and confidence scores before acting on an answer
- Live cluster diagnostics are optional (`ENABLE_LIVE_CLUSTER_TOOLS`) — still read-only

How to get there

- Surface: `/dashboard/ask` · hash `#ask`
- UI: Mission Control → **Console** panel → **Ask Zyra** (side rail, or ⌘K → “Ask Zyra”)

Operate from the console (UX)

1. Install optional extras: `pip install -e ".[knowledge]"` , start Qdrant, set `LLM_API_KEY` — see [Tutorial 14](#).
2. Open **Ask Zyra**, pick product / document-type filters, ask a question.
3. Review citations, confidence badge, and 📄 **copy MD** for Slack/Jira.
4. Header **knowledge lamp** shows ready / degraded / offline.

Without `[knowledge]` extras, Mission Control still works; Ask Zyra shows offline with install hints.

Related pages

- [Tutorial 14 — Ask Zyra](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

AI test

Purpose

Describe a goal; the autonomous browser agent drives the app toward it.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/ai-test`
- UI: Mission Control → **Journeys** → **AI test** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **AI test**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Flow test

Purpose

Multi-step user journey (English or step DSL) with optional login/session, video, and Playwright `trace.zip`.

When to use it

- Prove a critical path after deploy (signup, checkout, create VM, ...)
- Capture a single end-to-end video for stakeholders
- Reuse a session from [Auth & session](#)

How to get there

- Surface: `/dashboard/actions/flow`
- UI: Mission Control → **Journeys** → **Flow test** (or ⌘K → "Flow")

Operate from the console (UX)

1. Enter the app URL and either English ("go to products, click Schedule Demo, assert contact") or one step per line.
2. Optional: login user/password, reuse a saved session file, pick browser / device / throttle.
3. Keep **record video** on, then 🎬 **Run journey**.
4. Watch the live job panel; download video + trace when finished.

Step language (excerpt)

`goto` , `click` , `hover` , `fill` , `select` , `upload` , `download` , `dialog` , `iframe` , `drag` , `press` , `click` , `wait` , `wait until` , `assert` , `assert url` , `assert api` , `assert aria` , `assert not` , `assert count` , `assert value` .

CLI equivalent: `argus flow run <url> --steps file` .

5. **Empty / fail:** Check health, auth, and domain dependencies.
6. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [HAR record / replay](#)
 - [Import codegen](#)
 - [Auth & session](#)
 - [Getting Started](#)
 - [Page index](#)
-

HAR record / replay

Purpose

Record a HAR while browsing routes, or replay the UI against a captured HAR (offline / deterministic network).

When to use it

- Freeze API responses for flaky backends
- Diff UI behavior with vs without live network
- Share a network fixture with the team (`ZYVOR_HAR_PATH`)

How to get there

- Surface: `/dashboard/actions/har`
- UI: Mission Control → **Journeys** → **HAR record / replay**

Operate from the console (UX)

1. **Record** — set mode `record` , enter URL + routes (`/` , `/pricing` , ...). HAR path can auto-generate.
2. **Replay** — set mode `replay` , point at the HAR file, optional expect-text, optional “allow missing routes”.
3. Click  **Run** and review the live log.
4. CLI: `argus api har-replay <url> --mode record|replay --har out.har` .
5. **Empty / fail:** Check health, auth, and domain dependencies.
6. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Flow test](#)
 - [Route sweep](#)
 - [Getting Started](#)
 - [Page index](#)
-

Import codegen

Purpose

Paste Playwright codegen JS/TS and convert it into Zuvor Argus flow steps (optionally run immediately).

When to use it

- You already recorded a journey with `playwright codegen`
- You want flow DSL without rewriting clicks by hand

How to get there

- Surface: `/dashboard/actions/import-codegen`
- UI: Mission Control → **Journeys** → **Import codegen**

Operate from the console (UX)

1. Paste codegen output (`page.goto` , `getByRole().click` , `fill` , ...).
2. Optionally tick **run as flow** and provide the app URL.
3. Click  **Import** — inspect emitted steps; run if requested.
4. CLI: `argus test import-codegen script.js [--run --url ...]` .
5. Local headed record alternative: `npm run record-flow -- <url> out.flow.json` .
6. **Empty / fail:** Check health, auth, and domain dependencies.
7. **Success:** Live data loads; mutations complete without error toasts.

Related pages

- [Flow test](#)
 - [Getting Started](#)
 - [Page index](#)
-

Findings

Purpose

Collected issues from API, auth, live-data, vitals, and audit jobs with export/clear.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/findings`
- UI: Mission Control → **Operations** → **Findings** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Findings**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

QA Runs

Purpose

History table of QA runs with pass/fail chips and sparkline trends.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/runs`
- UI: Mission Control → **Operations** → **QA Runs** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **QA Runs**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Schedules

Purpose

Turn smoke, audit, ping, TLS, flow, route sweep, API, realtime, or vitals into a 5 min–6 h loop.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/schedules`
- UI: Mission Control → **Operations** → **Schedules** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Schedules**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Videos

Purpose

Browse and download recorded journey videos / traces from recent jobs.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/videos`
- UI: Mission Control → **Operations** → **Videos** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Videos**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Command palette

Purpose

⌘K / Ctrl-K or header **Search** — spotlight to jump to any action, panel, or Ask Zyra.

When to use it

- You know the action name but not which category panel it lives in
- Fast keyboard-driven navigation without scrolling the side rail

How to get there

- Surface: `/dashboard/command-palette`
- UI: press ⌘K (Ctrl-K) anywhere in Mission Control, or click header **Search**

Operate from the console (UX)

1. Open the palette from the keyboard or **Search** button.
2. Type to filter — e.g. "flow", "Ask Zyra", "TLS".
3. Enter to run or navigate; Esc to close.

Related pages

- [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Hero status

Purpose

Cluster/app health banner with pods, replicas, last QA run, pass rate, next schedule, and knowledge (Ask Zyra) stat tile.

When to use it

- First screen after sign-in — one glance at overall health
- Stat sub-lines explain offline cluster, empty run history, or knowledge install state

How to get there

- Surface: `/dashboard/hero` · hash `#overview`
- UI: Mission Control → **Overview** panel (default landing)

Operate from the console (UX)

1. Read the status lamp and headline (ALL SYSTEMS GO / DEGRADED / SYSTEMS DOWN / CLUSTER OFFLINE).
2. Scan stat tiles — pods/replicas show **No cluster connected** when kube is unavailable (not an error on a laptop).
3. **Knowledge** tile reflects Ask Zyra status; hover for install hints when offline.

Auto-refreshes every 5 s; press `r` to refresh now.

Related pages

- [Using the Dashboard](#)
 - [Mission Control](#)
 - [Ask Zyra](#)
 - [Page index](#)
-

Live job panel

Purpose

Live panel for the running job — macOS Terminal chrome, syntax-colored streaming log, per-test chips, Stop / download.

When to get there

- Appears automatically when any action card starts a job (one job at a time)
- Usually visible on **Overview** or the panel where you launched the job

How to get there

- Surface: `/dashboard/job-live`
- UI: start any action from a category panel or the ⌘K palette

Operate from the console (UX)

1. Watch the Terminal-style log and ✓/✗ case chips while the job runs.
2. Use **Stop** to cancel mid-flight.
3. Copy or download the log; when finished, use report / video / trace links in the result area.

Related pages

- [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Pods

Purpose

Pod cards with phase, restarts, and click-through logs; optional cluster events toggle.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/pods`
- UI: Mission Control → **Overview** → **Pods** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Pods**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from [.env.example](#).

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Workloads

Purpose

Deployment and CronJob strip for the argus namespace (when kube access is available).

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/workloads`
- UI: Mission Control → **Overview** → **Workloads** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Workloads**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from [.env.example](#).

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Mission Control

Purpose

Live Mission Control console — Apple-style side rail, dark theme, status hero, workloads, pods, category action panels, schedules, findings, and QA run history.

When to use it

- Your starting point for every QA action and cluster health view
- Prefer the **side rail** or **⌘K / Search** if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard`
- UI: sign in at `/login` when auth is enabled, then use the **side rail** or `#overview` hash URL

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. **Overview** — hero stat tiles, workloads, pods, QA runs.
3. **Pipeline / Visual / Quality / ...** — run jobs from category panels; one job at a time with a live Terminal-style log panel.
4. **Ask Zyra** — optional citation-first knowledge Q&A ([Tutorial 14](#)).
5. **Operations** — schedules, findings, run history, videos.
6. **⌘K** or header **Search** — command palette to jump anywhere.

Use the **theme toggle** (moon/sun) for light mode; **Collapse** on the rail for icons-only.

If the console stays idle or errors, hit `GET /health` , confirm `argus serve` (or `systemd argus.service`) is up, and re-check env from [.env.example](#).

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Ask Zyra](#)
 - [Page index](#)
-

Login

Purpose

Mission Control sign-in when `DASHBOARD_PASSWORD` is set. Uses the same dark/light design system as the dashboard (charcoal default, blue accent buttons). Argus Enterprise uses Keycloak/OIDC with demo accounts `demo / demo` and `ssouser / Sso@321` (see the Enterprise SSO chapter in this manual).

For **Argus Enterprise** (Watchfloor) full setup steps, open [Enterprise SSO / OIDC](#) — that product does not use `DASHBOARD_PASSWORD`.

When to use it

- You opened `/dashboard` and were redirected to `/login`
- You need to confirm Mission Control auth before running cards that read cluster logs or artifacts

How to get there

- Surface: `/login`
- UI: open `/dashboard` when auth is enabled (redirect), or go directly to `/login`

Operate from the console (UX)

Mission Control (community)

1. Ensure `DASHBOARD_PASSWORD` (and optional `DASHBOARD_USER`, default `admin`) is set.
2. Open `/login`, enter username + password (lab defaults are often `admin / Admin@321` — rotate for anything exposed).
3. Optional: check **Remember me** for a longer session (30 days vs 12 h).
4. After success you land on `/dashboard`. `/health` and `/webhook/github` stay open.

Theme preference (`zyvor-theme` in browser storage) is shared with the dashboard after sign-in.

Argus Enterprise (Watchfloor)

1. Claim the fresh install with the one-time setup token from app logs.
2. On the login screen, choose the SSO / Keycloak provider (or the local email/password form if local auth is enabled).
3. Demo accounts after a default Keycloak install:
 - `demo / demo`
 - `ssouser / Sso@321`
4. Verify with `scripts/test-login.sh https://argus.example.com demo demo`.

Full steps (Helm, BYO OIDC, local auth, troubleshooting): [Enterprise SSO / OIDC](#).

If Mission Control login fails, hit `GET /health`, confirm `argus serve` is up, and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
- [Admin basics](#)
- [Enterprise SSO / OIDC](#)
- [Using the Dashboard](#)

- [Mission Control](#)
 - [Page index](#)
-

Create from English

Purpose

Turn an English description into a one-off test (LLM when keyed, heuristic otherwise).

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/create`
- UI: Mission Control → **Pipeline** → **Create from English** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Create from English**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Discover coverage

Purpose

Scan repo code/docs for untested routes and pages, then propose coverage.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/discover`
- UI: Mission Control → **Pipeline** → **Discover coverage** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Discover coverage**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Generate tests

Purpose

Generate Playwright tests from a product spec (local path or GitHub).

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/generate`
- UI: Mission Control → **Pipeline** → **Generate tests** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Generate tests**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Run tests

Purpose

Run ► Smoke (optional grep/shard) or ► Full LangGraph pipeline from local or GitHub specs.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/run-tests`
- UI: Mission Control → **Pipeline** → **Run tests** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Run tests**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Load test

Purpose

Simple concurrency load test for latency under pressure.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/loadtest`
- UI: Mission Control → **Probes** → **Load test** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Load test**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Uptime ping

Purpose

HTTP uptime / latency checks across a list of URLs.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/ping`
- UI: Mission Control → **Probes** → **Uptime ping** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Uptime ping**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Probes

Purpose

One-shot network & security probes: redirects, headers, cookies, robots, exposed paths, API, sitemap, DNS, CORS, compression.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/probes`
- UI: Mission Control → **Probes** → **Probes** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Probes**, then start the action and watch the live job panel (✓/X chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

TLS check

Purpose

Certificate and DNS health for a hostname.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/tls`
- UI: Mission Control → **Probes** → **TLS check** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **TLS check**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Site audit

Purpose

Crawl pages for a11y (axe), broken links, SEO, console, perf, and security headers.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/audit`
- UI: Mission Control → **Quality** → **Site audit** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Site audit**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Crawl & test

Purpose

Crawl an arbitrary site and validate every discovered page.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/crawl`
- UI: Mission Control → **Quality** → **Crawl & test** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Crawl & test**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Flaky check

Purpose

Re-run the suite N times and surface unstable tests.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/flaky`
- UI: Mission Control → **Quality** → **Flaky check** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Flaky check**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Web Vitals

Purpose

Measure Core Web Vitals (LCP / CLS / INP) with optional device and network throttle.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/vitals`
- UI: Mission Control → **Quality** → **Web Vitals** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Web Vitals**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Attack chain

Purpose

Chains exploit-PoC steps via an LLM planner to confirm a multi-step escalation path — same sandbox and opt-ins as Exploit PoC.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/attack-chain`
- UI: Mission Control → **Security** → **Attack chain** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Attack chain**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from [.env.example](#).

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Cloud pentest

Purpose

Credentialed AWS/GCP/Azure CLI enumeration of a described finding in the sandbox — same opt-ins and credential-reference rules as Host pentest.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/cloud-pentest`
- UI: Mission Control → **Security** → **Cloud pentest** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Cloud pentest**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

CVE lookup

Purpose

Read-only: fingerprints tech/versions and checks them against OSV.dev. No PoC is generated or run — requires a security engagement.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/cve-lookup`
- UI: Mission Control → **Security** → **CVE lookup** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **CVE lookup**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from [.env.example](#).

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Exploit PoC

Purpose

LLM-generated, non-destructive verification of a described finding, executed in a locked-down Kubernetes sandbox — requires an exploit-tier engagement and an execution opt-in.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/exploit-poc`
- UI: Mission Control → **Security** → **Exploit PoC** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Exploit PoC**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from [.env.example](#).

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Host pentest

Purpose

Credentialed SSH enumeration of a described finding via paramiko in the sandbox — needs a third, independent credentialed-pentest opt-in; credentials are always env-var references, never raw values.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/host-pentest`
- UI: Mission Control → **Security** → **Host pentest** (side rail panel, or ⌘K / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Host pentest**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

LLM red-team

Purpose

Attacker/judge adversarial-prompt battery against Ask Zyra — prompt injection, system-prompt leak, excessive agency, jailbreak, PII/secret leak — requires a security engagement.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/llm-redteam`
- UI: Mission Control → **Security** → **LLM red-team** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **LLM red-team**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Misconfig scan

Purpose

Tech/version fingerprinting, wordlist-driven path discovery, security-header value grading, and DNS hygiene checks — requires a security engagement.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/misconfig-scan`
- UI: Mission Control → **Security** → **Misconfig scan** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Misconfig scan**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Security engagements

Purpose

Create/revoke the admin-issued, target-scoped authorization required before running misconfig scan, CVE lookup, or LLM red-team.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYVOR_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/security-engagements`
- UI: Mission Control → **Security** → **Security engagements** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Security engagements**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Compare URLs

Purpose

Side-by-side visual diff of two URLs (e.g. staging vs production).

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/compare`
- UI: Mission Control → **Visual** → **Compare URLs** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Compare URLs**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Visual regression

Purpose

Pixel-compare visual snapshots against baselines; optionally update baselines.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/regression`
- UI: Mission Control → **Visual** → **Visual regression** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Visual regression**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Route sweep

Purpose

Screenshot many routes (or auto-crawl) and diff against route-sweep baselines.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/route-sweep`
- UI: Mission Control → **Visual** → **Route sweep** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Route sweep**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Screenshot

Purpose

Capture desktop / tablet / mobile screenshots of any URL.

When to use it

- Open this card when the job matches the purpose above
- Prefer **Mission Control** (`/dashboard`) and the **⌘K** command palette if you are unsure where to start
- Confirm `ZYV0R_BASE_URL` , dashboard auth, and that Playwright browsers are installed if runs fail immediately

How to get there

- Surface: `/dashboard/actions/screenshot`
- UI: Mission Control → **Visual** → **Screenshot** (side rail panel, or **⌘K** / Ctrl-K Search)

Operate from the console (UX)

1. Open `/dashboard` (sign in at `/login` when `DASHBOARD_PASSWORD` is set).
2. Fill the card fields for **Screenshot**, then start the action and watch the live job panel (✓/✗ chips, Stop, download log).
3. After success, check **Findings**, **QA Runs**, and any video / report links the card produces.
4. Turn recurring checks into a **Schedule** (5 min – 6 h) when you want continuous monitoring.

If the card stays idle or errors, hit `GET /health` , confirm the webhook/dashboard process is up (`argus serve`), and re-check env from `.env.example`.

Related pages

- [Getting Started](#)
 - [Using the Dashboard](#)
 - [Mission Control](#)
 - [Page index](#)
-

Zyvor Argus — Complete page index

Every Mission Control surface and action card.

43 routes

Regenerate: `node scripts/customer-docs/generate-page-index.mjs`

Overview

Page	Route	Purpose	Guide
Mission Control	<code>/dashboard</code>	Live Mission Control console — side-rail navigation, dark theme, status hero, workloads, pods, category action panels, schedules, findings, and QA run history.	Open
Login	<code>/login</code>	Mission Control sign-in — same dark/light design system as the dashboard; DASHBOARD_PASSWORD gate. Argus Enterprise uses Keycloak SSO (demo/demo) — see customer/enterprise-sso.md.	Open
Hero status	<code>/dashboard/hero</code>	Cluster/app health banner with pods, replicas, last QA run, pass rate, and next scheduled smoke.	Open
Workloads	<code>/dashboard/workloads</code>	Deployment and CronJob strip for the argus namespace (when kube access is available).	Open
Pods	<code>/dashboard/pods</code>	Pod cards with phase, restarts, and click-through logs; optional cluster events toggle.	Open
Live job panel	<code>/dashboard/job-live</code>	Live panel for the running job — macOS Terminal chrome, syntax-colored log, per-test chips, Stop / download.	Open
Command palette	<code>/dashboard/command-palette</code>	⌘K / Ctrl-K or header Search — spotlight to jump to any action or Ask Zyra.	Open

Console

Page	Route	Purpose	Guide
Ask Zyra	<code>/dashboard/ask</code>	Ask Zyra — citation-first product knowledge Q&A (optional Qdrant RAG); read-only, no cluster mutations.	Open

Pipeline

Page	Route	Purpose	Guide
Run tests	<code>/dashboard/actions/run-tests</code>	Run ► Smoke (optional grep/shard) or ► Full LangGraph pipeline from local or GitHub specs.	Open
Generate tests	<code>/dashboard/actions/generate</code>	Generate Playwright tests from a product spec (local path or GitHub).	Open
Discover coverage	<code>/dashboard/actions/discover</code>	Scan repo code/docs for untested routes and pages, then propose coverage.	Open
Create from English	<code>/dashboard/actions/create</code>	Turn an English description into a one-off test (LLM when keyed, heuristic otherwise).	Open

Visual

Page	Route	Purpose	Guide
Visual regression	/dashboard/actions/regression	Pixel-compare visual snapshots against baselines; optionally update baselines.	Open
Compare URLs	/dashboard/actions/compare	Side-by-side visual diff of two URLs (e.g. staging vs production).	Open
Screenshot	/dashboard/actions/screenshot	Capture desktop / tablet / mobile screenshots of any URL.	Open
Route sweep	/dashboard/actions/route-sweep	Screenshot many routes (or auto-crawl) and diff against route-sweep baselines.	Open

Quality

Page	Route	Purpose	Guide
Site audit	/dashboard/actions/audit	Crawl pages for a11y (axe), broken links, SEO, console, perf, and security headers.	Open
Flaky check	/dashboard/actions/flaky	Re-run the suite N times and surface unstable tests.	Open
Web Vitals	/dashboard/actions/vitals	Measure Core Web Vitals (LCP / CLS / INP) with optional device and network throttle.	Open
Crawl & test	/dashboard/actions/crawl	Crawl an arbitrary site and validate every discovered page.	Open

Probes

Page	Route	Purpose	Guide
Uptime ping	/dashboard/actions/ping	HTTP uptime / latency checks across a list of URLs.	Open
Load test	/dashboard/actions/loadtest	Simple concurrency load test for latency under pressure.	Open
TLS check	/dashboard/actions/tls	Certificate and DNS health for a hostname.	Open
Probes	/dashboard/actions/probes	One-shot network & security probes: redirects, headers, cookies, robots, exposed paths, API, sitemap, DNS, CORS, compression.	Open

Journeys

Page	Route	Purpose	Guide
Flow test	/dashboard/actions/flow	Multi-step user journey (English or step DSL) with optional login/session, video, and trace.	Open
HAR record / replay	/dashboard/actions/har	Record a HAR while browsing routes, or replay the UI against a captured HAR.	Open

Page	Route	Purpose	Guide
Import codegen	<code>/dashboard/actions/import-codegen</code>	Paste Playwright codegen JS/TS and convert it into flow steps (optionally run).	Open
AI test	<code>/dashboard/actions/ai-test</code>	Describe a goal; the autonomous browser agent drives the app toward it.	Open

API

Page	Route	Purpose	Guide
API contract	<code>/dashboard/actions/api-contract</code>	Validate REST endpoints against an OpenAPI schema (Forge preset available).	Open
Live data	<code>/dashboard/actions/realtime</code>	Assert WebSocket / SSE streams and optional live-region updates.	Open
Auth & session	<code>/dashboard/actions/auth</code>	Login → reusable session file → logout / expiry / negative auth checks.	Open

Security

Page	Route	Purpose	Guide
Security engagements	<code>/dashboard/actions/security-engagements</code>	Create/revoke the admin-issued, target-scoped authorization required before running any deeper security testing job.	Open
Misconfig scan	<code>/dashboard/actions/misconfig-scan</code>	Tech/version fingerprinting, wordlist-driven path discovery, security-header value grading, and DNS hygiene checks — requires a security engagement.	Open
CVE lookup	<code>/dashboard/actions/cve-lookup</code>	Read-only: fingerprints tech/versions and checks them against OSV.dev. No PoC is generated or run — requires a security engagement.	Open
LLM red-team	<code>/dashboard/actions/llm-redteam</code>	Attacker/judge adversarial-prompt battery against Ask Zyra — prompt injection, system-prompt leak, excessive agency, jailbreak, PII/secret leak — requires a security engagement.	Open
Exploit PoC	<code>/dashboard/actions/exploit-poc</code>	LLM-generated, non-destructive verification of a described finding, executed in a locked-down Kubernetes sandbox — requires an exploit-tier engagement and an execution opt-in.	Open
Attack chain	<code>/dashboard/actions/attack-chain</code>	Chains exploit-PoC steps via an LLM planner to confirm a multi-step escalation path — same sandbox and opt-ins as Exploit PoC.	Open
Host pentest	<code>/dashboard/actions/host-pentest</code>	Credentialed SSH enumeration of a described finding via paramiko in the sandbox — needs a third, independent credentialed-pentest opt-in; credentials are always env-var references, never raw values.	Open
Cloud pentest	<code>/dashboard/actions/cloud-pentest</code>	Credentialed AWS/GCP/Azure CLI enumeration of a described finding in the sandbox — same opt-ins and	Open

Page	Route	Purpose	Guide
		credential-reference rules as Host pentest.	

Operations

Page	Route	Purpose	Guide
Schedules	/dashboard/schedules	Turn smoke, audit, ping, TLS, flow, route sweep, API, realtime, or vitals into a 5 min–6 h loop.	Open
Findings	/dashboard/findings	Collected issues from API, auth, live-data, vitals, and audit jobs with export/clear.	Open
QA Runs	/dashboard/runs	History table of QA runs with pass/fail chips and sparkline trends.	Open
Videos	/dashboard/videos	Browse and download recorded journey videos / traces from recent jobs.	Open

Related

- [Customer docs home](#)
- [Page-by-page guides](#)